Programming Project II

Compiler Register Allocation Algorithms

Spring 2026

***Due Date:** May 24, 2026, at 23:59 (PT time)*

1. Objectives

This first programming assignment aims at exposing you to realistic implementation of algorithmic solutions presented in class and in particular **approximation and heuristic approaches** in the context of NP-complete problems. Key objectives include:

- Developing teamwork skills by working in groups of **2-3 students (3 preferred)**, fostering interpersonal and project management skills.
- Conceive and implement a compiler register allocation tool that can be used in the context of a program intermediate representation as part of the compiler back-end.
- Preparing and delivering a **concise demo presentation** to showcase the functionalities and interfaces developed, while enhancing your ability to prioritize and communicate essential information effectively. This will require you to be **succinct**, **time-conscious**, and focused on what is essential (and what is not).

This document describes the motivation of this project, the expected interface followed by a description of the problem statement and description of the demo you are expected to present. The problem statement includes a description of each task (alongside the corresponding grading). Lastly, we provide specific turn-in instructions you need to follow. **Recall, the deadline is May 24, 2026 at 23:59.**

2. Problem Motivation

In this project, you will create a tool to explore several register allocation strategies that can be used as part of a compiler back-end. Your algorithm implementation will operate on an intermediate representation that consists of 3-address instructions used in the mapping of high-level programing constructs to sequences of these instructions. In general terms register allocation is responsible to map high-level programming language variables to memory and/or to registers for enhanced performance. It is among the most important transformations a compiler can do. As an illustrative example, consider the generic imperative programming language source code below (a). A possible translation into 3-address intermediate-level instructions is also shown (b) and a trivial mapping of variables to register is shown in (c). In this example, and ignoring the temporary variable `t0`, the variables `sum` and `res` map to register `r0`; variable `i` to register `r1`; variable `n` to `r2`; so a total of 3 registers is required. Notice here that the last line appears to be incorrect. In reality, as described below, the same register can be used to hold the values of different variables if they are not subsequently required. Beyond the last statement "`res = sum`" there is really no need to use the value of `sum` and hence the corresponding register can be "released" and used for the `res` variable.

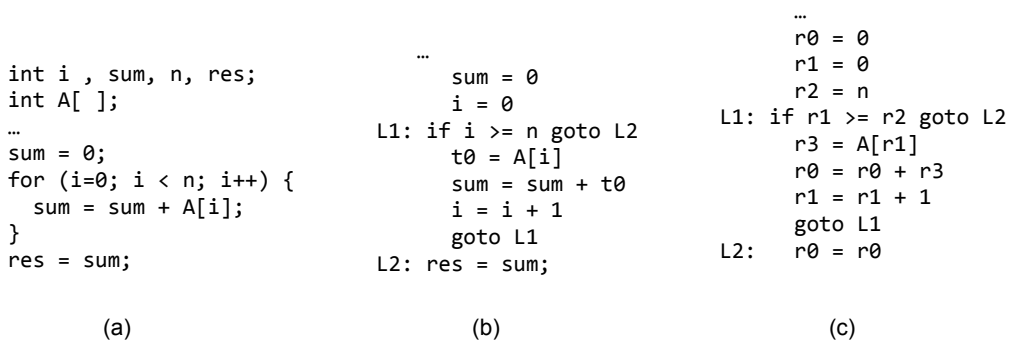


Figure 1. Simple example of intermediate code generation from high-level source code and corresponding possible register allocation representation.

2.1. Intermediate Code Representation

A simple intermediate representation that can be used to support the mapping of many high-level imperative programming languages is the 3-address instruction representation described in detail in Appendix A. Here, instruction can be divided into 3 categories, namely:

- **Basic Arithmetic/Logic Instructions:** These are generic two operand instructions such as "`a = b op c`" where `op` can be an arithmetic or logic operation defined over integers, Boolean or even real numbers. Obviously, this extends to unary operations such as logical or arithmetic complement as well as the simple assignment of variables such as "`a = b`" (see memory operations below).

- **Control-Flow Instructions:** These are generic control-transfer instructions such as (un)conditional jumps and `call`/`ret` instructions. The conditional jump have the simple form of "`if pred goto L`" where `pred` is a simple predicate of the form "`a op b`" where types are compatible with the relational operator. Here `L` is a label of an instruction. More complex predicate can be developed by a sequence of instructions using logic operator such as `and`, `or` and `not`. In the case of the `call` instructions arguments are set up on a stack (deliberately ill-defined in this representation) where values are stored (using the `putparam` instructions) and retrieved (using the `getparam` instruction). The `call` instruction itself has two parameters and can have a return value associated with it. In addition, it contains a numeric number indicating the number of arguments and correspondingly the number of `putparam` instructions that precede the `call` instruction itself. The `ret` instruction may have a single operand for the case of procedure that returns a single value (in some languages there are called functions rather than procedures). The example in Figure 2 below depicts an illustrative example.

- **Memory/Stack Operations:** Direct manipulation of (scalar) variables are handled directly by using its identifier. So, the assignment "`a = b`" is implemented as a load of the memory contents associated with the symbol `b` into an internal auxiliary processor register, followed by a store of that value to the memory location associated with the symbol `a`. In addition, to these scalar operations, we also support single-dimension array operations of the form "`x = A[t0]`" or "`A[t1] = y`" where `x`, `y`, `t0` and `t1` are scalar variables (some of which intermediary variable inserted by the compiler itself) and `A` is a symbol associated with an array variable. Multi-dimensional arrays are not supported as they can be internally converted into linear structures with the appropriate indexing or memory address indirection.

Figure 2 below depicts an illustrative example of the mapping of a program (akin to C) with a function definition and the corresponding invocation in the described 3-address code representation.

```
void main(){
  ...
  i = 0;
  do
  a = A[i];
  y = myAdd(a, i+1);
  if y > 0 then
    break;
  i = i + 1;
  while (i < len);
  ...
}

int myAdd(x,z){
  return x+z;
}
```

(a)

```
main:
    ...
    i = 0
L1: a = A[i]
    t1 = a
    t2 = i + 1
    putparam t1
    putparam t2
    y = call myAdd, 2
    if y > 0 goto L2
    i = i + 1
    goto L1
L2: ...


myAdd:
     getparam z
     getparam x
     t3 = x + z
     ret t3
```

(b)

Figure 2. Simple example of a loop and function invocation in the 3-address representation description.

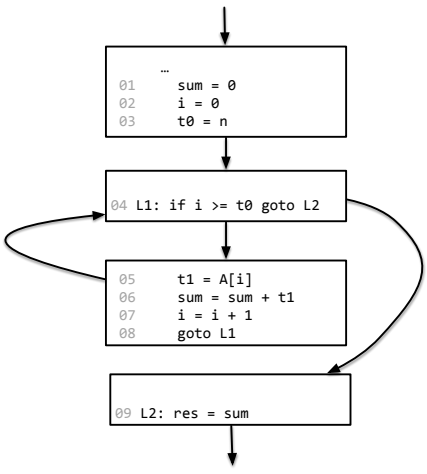
A final note on type safety. In your inputs it is assumed that the 3-address code you are given is type safe. In fact, there are no variables declaration in this representation as these assumed to be handled in other phases of the compiler. As such, you are concerned with variable and procedure (label) symbols only.

2.2. Control-Flow-Graphs (CFG)

Based on the control-flow, compilers internally build what is called a Control-Flow-Graph (CFG) where the nodes consist of "uninterrupted" or maximal sequences of instructions such that if the first instruction of the sequence all the other instructions in the basic block sequence **must** execute. Such sequences define what is called a basic block (BB). As such, the flow of control can only start at the first instruction of the basic block and terminate at the last instruction of each basic block. Edges in the control flow represent explicit or implicit control transfer and include unconditional, conditional jumps as well as call and return statements. Figure 3 below depicts the CFG corresponding to the 3-address code in Figure 1 b) where we have labelled each instruction with a symbolic line number (in gray).

```
   …
01     sum = 0
02     i = 0
03     t0 = n
04 L1: if i >= t0 goto L2
05     t1 = A[i]
06     sum = sum + t1
07     i = i + 1
08     goto L1
09 L2: res = sum
```

(a)



(b)

Figure 3. Control-Flow-Graph (CFG) representation of 3-address code in Figure 1.c).

An algorithm to define each of the basic blocks is as shown in Figure 4 below. It follows the notion that every basic block begins with the "potential" target of a `goto` instruction and if the first instruction of a basic block is executed, then all the remaining instructions are.

```
1. for all instruction in the sequence do
  a. The first instruction of the sequence is a "leader" instruction
  b. Any instruction that is the target of a (un)conditional goto is a leader
  c. Any instruction that immediately follows an (un)conditional goto is a leader
2. for each leader instruction identified in 1. above do
  a. create a new basic block B
  b. insert in B the leader instruction and all the instructions that follow it up
     to (and excluding) the next leader instruction.
3. for all identified basic blocks
  a. define its outgoing edges from their last instruction to leader instructions
     of other basic blocks.
```

Figure 4. Algorithm to uncover the basic blocks of a Control-Flow-Graph (CFG) representation.

2.3. Register Assignment and Allocation

Technically, the compiler phase of code generation from intermediate program representation (or IR) deals with two complementary issues, namely, register assignment and register allocation. Register assignment deals with the issue of deciding which program-level (or intermediate representation level) should be mapped to (virtual) register, leaving the unmapped variable to reside in memory. Register allocation deals with the problem of specifically mapping each variable to one of the K available physical registers.

The astute reader might understand that a given variable may in fact be mapped to distinct physical registers as long as it is only mapped to one of them at a given point in time during the execution. In this project you are asked to develop what is called a "global" register allocation algorithm that combines these two issues found in register allocation.

2.4. Variable Live Range and Webs

At the core of a "global" register allocation (assignment and mapping) algorithm is the concept of "live range" which is defined by the program execution path defined from the execution point a variable (temporary or not) is first defined and the last execution point where the corresponding value is no longer needed. Notice that a variable may have distinct live ranges as through the execution of the program, corresponding to distinct value the variable may assume.

The Figure 5 below depicts a CFG and the live range of three variables, namely `i`, `t1` and `sum`. Observe for instance, that live ranges are merged at selected points of the execution whenever two or more possible values of the variable `i` can be read at the execution point at line `04`.

In this assignment you will need to understand this notion of liveness and determine the execution points and therefore execution path along which a variable value may still be used in the future (i.e., along that path). To make this assignment feasible in its time frame, you will be given an indication of the live ranges of each variable in the corresponding input program, based on which, you need to construct the live webs.

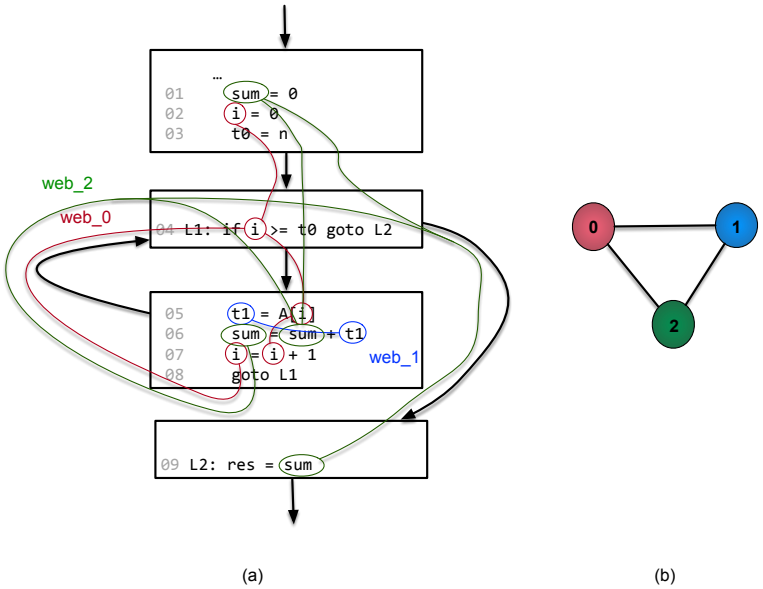


Figure 5. Example of live ranges/web for variables `i` , `t1` and `sum` (a) and the corresponding (colored) interference graph (b)

Based on the live range of each variable, compilers construct live web, which is nothing more that the union of the live ranges of a given variable, should they overlap at any point of the execution of the program. For the particular case above the lives ranges of the variable `i` are "merged" at the execution point on line `04` as two definitions (therefore potential values) can be used (or read) at that point, namely, the value defined on line `02` as well as the value define on line `07` is used on line `04`.

2.5. Interference Graph

At the core of any register allocation algorithm is the observation that what needs to be "captured" in a register is not a variable per se, but its value. As such, a register can be "allocated" different variables during the execution of a program as long as not simultaneously. In fact, a register can be allocated different values of different variables, but never simultaneously.

To capture this concept, what are called "global" register allocation algorithms, try to map, not variables to registers, but live webs to registers. This mapping must take into account the ranges of the webs so that a given register cannot hold values of more than one web at a time. To enforce this constraint, the algorithm constructs a web interference graph (or interference graph for short) where nodes of this graph correspond to webs and the edges between to nodes are defined if there is at least one execution point where both webs coexist, i.e., they interfere. Figure 5 (b) depicts the interference graph for the three webs (which in this particular case coincide with the live ranges of the corresponding variables.

There is a subtlety regarding the notion of interference as one needs to be precise about the last execution point where a value is still needed (as there is a potential future use). Two webs, one (say A) that starts at a given instruction where another web (say B) ends, do not interfere is A begin at a given instruction due to the definition of a variable and B ends at the same instruction due to a use of a variable. Figure 6 below illustrates the case focusing exclusively on a single basic block, showing web that interfere and other that do not.

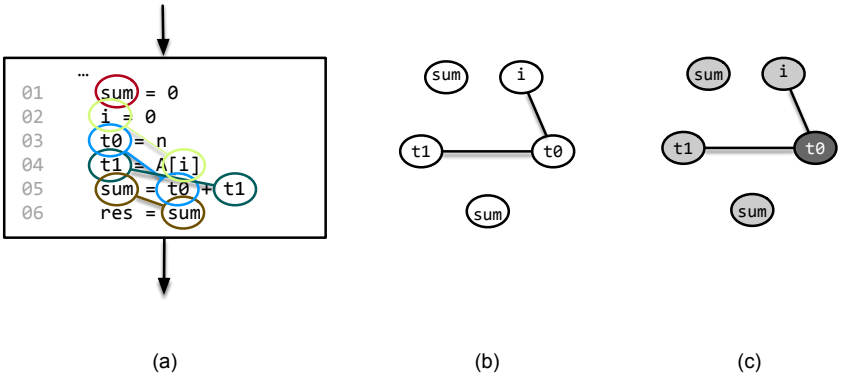


Figure 6. Illustration of web interference and corresponding minimally colored interference graph.

In particular, the webs corresponding to the variables `t0` and `t1` do interfere, but neither of these interfere with the web from the variable `sum`. Except for the `res` variable it is assumed here that none of the variables is "live" at the exit of the basic block, that is, their values are not needed. As also shown above, the minimum number of colors required to color this graph, and therefore the minimum number of registers is 2.

2.6. Register Allocation - Graph Coloring

The problem of register allocation, now, taunt amounts to the problem of graph coloring where a no two nodes that are connected by an edge can have the same color. In the register allocation parlance, a color will correspond to a physical register, and the goal of the compiler will be to find the minimum number of colors that a given interference graph can be colored, *i.e.*, the minimum number of registers that can be used for the computation at hand. For the illustrative case in Figure 5 above on page 5 the minimum number of colors is 3 as in this case the 3 graph nodes forma clique of size 3.

3. Problem Description and Interface

In this project you are asked to develop a suite of algorithms for register allocation through graph coloring in which the minimum number of colors, and therefore of physical registers is sought. As briefly described in class, the problem of graph coloring has been shown to be NP-complete. So, as a result, you are expected to use a simple graph coloring heuristic algorithm as described below and develop others of your own for selected variants of the problem.

In this project, you will be given a representation of the live ranges for each variable with specific formats as described in the next section. The focus will then be on combining the live ranges to form a web and then an interference graph that will be the input for your register allocation (i.e., graph coloring). Several approaches are possible and in the context of limited registers you will need to make several algorithmic decisions to optimize (at least try) to make a valid register assignments with the lowest cost.

As such, the key aspects of the project work include:

a. Understanding the webs for all the variables of a given code. These are based on the live ranges provided as part of the input file, but you need to pay attention to the notion of interference and the way the ranges are described to accurately build the interference graph.
b. Definition of the corresponding webs and interference graph.
c. Determine the minimum number of colors the graph can be colored.

While steps a. and b. above have been described above, graph coloring is a hard problem for which there are various heuristics.

3.1. Description of Variables and Live Ranges

As the main input files of your project, you are given a mapping of variables to their live ranges. This text file has the structure shown below which include an indication of each variable the program manipulates and the corresponding live ranges.

The description of a variable is rather trivial, as it consists of a unique string. Its live ranges, however, can be many as the variable may be manipulated at various sections of the program. As such, we describe the live range for each variable as a list of program line numbers where the variable is alive. As a variable can have many live ranges, we can have various lines the input file one per live range of the same variable. Clearly, for a given variable, there must be at least one live range that begins with the definition of the variable's value which is indicated by the line number with a "+" character. Also, there must be a live range with a line number followed by the "-" character. This "-" indication reflects the fact that the live range for the corresponding variable ends at that line, that is, any execution beyond that point will either not use the variable anymore, or will first redefine the variable before using it. This "+" designation indicates that the specific live range begins with an assignment to the variable as the Left-Hand-Side (LHS) at a specific program line (corresponding to a write) or ends with a read of that variable as the Right-Hand-Side (RHS) at that specific program line.

Note that because live ranges intersect, the program line numbers corresponding to this intersection are themselves live ranges but there are not necessarily assignments or uses of that variable in them. As such, these live ranges do not begin with a live number with a "+" indication or with a "-" indication. It is even possible that a variable has a live range ending on a given line and another live range starting on that line as is the case of the presence of a statement of the form "`i=i+1`". In this case you should fuse the two live ranges and consider the second live range an extension of the first one. The example input file in Figure 7 below includes 2 variables, respectively, "`sum`" and "`i`" with various live ranges. Line numbers are separated by commas.

```
# this line is just a comment to be ignored
sum: 7+,8,9,10-
i: 1+,2,3,4,5,6-
i: 9+,10,11,12-
i: 12+,13,14-
i: 20+,11,12-
```

Figure 7. Example of a Live Ranges input file

Another example input file in Figure 8 below includes 2 variables, respectively, "`x`" and "`y`" with various live ranges. Line numbers are separated by commas. Here, there is a single "web" corresponding to the variable "`x`" as aggregately these live ranges shared execution points (program lines) that are common. A simple greedy algorithm can be used to merge live ranges into webs.

```
# this line is just a comment to be ignored
y: 7+,8,9,10-
x: 1+,2,3,4,7
x: 7,8
x: 8,9-
x: 8,11,12,13-
```

Figure 8. Example of a Live Ranges input file.

.

3.2. A Simple Graph Coloring Algorithm

A simple graph coloring algorithm will attempt to color a given graph with N colors is described in Figure 9. Here N is a parameter of the algorithm, and the overall idea is to find the minimum N for which a given graph is still colorable. Notice, obviously, that for N sufficiently large it is always possible to color a graph. In the worst case with N equals the number of nodes, each node would have its distinct color making the graph trivially colored.

```
int greedyGraphColoringAlgorithm(Graph G, int N)
  // 1. Clearing cliques
  while graph is not empty do
    for all nodes n in G do
      if degree(n) < N then
        remove n from G
        push n onto a stack S
    if all nodes remaining in G have degree >= N then
       select a node K to spill (no color assigned to this node at all)
       remove K from the graph
  // 2. Coloring phase
  while stack is not empty do
    n = pop node from stack S
    assign it a color that is different from the color of its neighbors in G
    // since degree of n is < N, a color should always exist.

  return N and color assignment
```

Figure 9. A simple greedy graph-coloring algorithm.

Obviously, by spilling enough nodes in step 1 it is always possible to color the graph with N colors. Since the focus in on finding the lowest value of N, you can use some heurists to scan N from a low number to a highest number where N is clearly upper bounded by the max degree of any node in G. Also, in terms of the implementation, "removing" a node in step 1 may not be implemented as a physical removal of the node, but rather, "disabling" it from the graph adjusting the incoming and outgoing edges. Alternatively, you may want to duplicate the graph and use the auxiliary graph to removing nodes from.

3.3. Alternative Graph Coloring Algorithm

In addition to the simple graph coloring algorithm just described, you are free to explore alternatives heuristics, which could explore the degree of the nodes and or specific nodes with lower degree. In addition, explore the use of metrics such as the planarity of a graph, partitioning of its nodes, articulation edges/nodes that attempt for the selected input cases efficient algorithms.

In this work you are also asked to explore the possibility of some of the webs not being assigned a register. In this case, the interference graph is simplified by removing the corresponding web's node and its incident edges, and the coloring is attempted with a "reduce" graph. This is called "web spilling" as the web's corresponding value of therefore committed to memory. Alternatively, you can split (or slice) one or more webs in two or more sections creating "derived" webs, in what is called "web splitting". In this case, the connectivity of the interference graph is possibly reduced allowing for its coloring with the maximum number of provided registers. In this case at the boundaries of the web spilling the compiler will have to insert instructions to save and restore its value to and from memory.

3.4. Input

In addition to the ranges input file, the registers input file is also required as it provides the maximum number of registers that can be used. A simple text file format shown below in Figure 10 will specific the integer number of registers to be used to be named **`r0`**, through **`rk`** with **`k = N-1`**.

```
# comment line
registers: N
# algorithm variants: basic, splitting and spilling each with a numeric parameter
algorithm: basic
```

Figure 10. Input file template.

3.5. Output

The output should follow a simple human-readable format indicating the number of registers used and the webs to which they associated with. As such, you need to output the webs in a specific format assigning a numeric number to which web which must be the same as the corresponding register. Note that in some cases you will not be able to allocate a register to a web. In this case, the web will have no register associated with but instead the character "`M`" indicating that the web will be "allocated" to memory.

```
# Total number of webs followed by the listing of the program points of each one
# program points in each web are sorted in ascending order
webs: 4
web0: 1+,2,3,4,5,6-
web1: 9+,10,11,12-,20+
web1: 12+,13,14-
web2: 7+,8,9,10-
# Total number of registers used, followed by assignment to webs
registers: 2
r0: web0
r0: web1
r0: web2
r1: web3
```

Figure 11. Small output file with illustrative web to register assignment.

Should the allocation not be possible the number of actual registers used should be 0 and the webs should all be assigned to memory indicating the **`M`** assignment. Figure 12 below depicts an example of such output case. Also, in this scenario you program should issue a warning to the console simply stating that the assignment to the provided number of registers was not possible.

```
# Total number of webs followed by the listing of the program points of each one
# program points in each web are sorted in ascending order
webs: 3
web0: 1+,2,3,4,5,6-,9-,10,11,12-,20+
web1: 12+,13,14-
web2: 7+,8,9,10-
# Total number of registers used, followed by assignment to webs
registers: 0
M: web0
M: web1
M: web2
```

Figure 12. Small output file with illustrative of infeasible register allocation.

4. Organization of Work

To facilitate your work, we have structured the functionalities that you are expected to develop as follows.

4.1. Basic Application Definition and Set-Up

[T1.1: 1.0 point] Develop the Assignment Tool. The first task will be to create a simple command-line menu that exposes all implemented functionalities in a user-friendly manner. This menu will also serve as a key component for showcasing the work you have developed in a short demo at the end of the project. When displaying the results, show all relevant details, including the register allocation. The program should handle exceptional cases/errors properly such as invalid ranges or parameter settings.

IMPORTANT: You should also provide a simple batch or command line interface where your program can be executed through a script. In this case the order of the arguments to your executable file should be as: `myProg -b ranges.txt registers.txt allocation.txt` where the "`-b`" indicates an execution in batch mode and the last parameter is the output text file. Error messages are to be directed to the console or in Unix the standard error file.

[T1.2: 1.0 point] Read and Parse the Input Data. You will need to develop basic functionality (accessible through your menu) to read and parse the provided data set files. This functionality will enable you (and the eventual user) to indicate the input files with the live ranges and register parameter settings. In addition, the indication of the output file should also be given.

With the extracted information, you should create one (or more) appropriate data structures upon which you will carry out the requested tasks. **This data structure must be based on the data structure described in the TP lectures for graphs, for which it may make a small number of modest additions.** You must use the provided graph structure as the primary representation of the interference graph, and you may use other auxiliary structures as needed to facilitate your implementation. Any alterations made to the original graph should be clearly justified in your presentation, ensuring that your modifications are sensible and enhance the application of the required algorithms.

[T1.3: 2.0 points] Documentation and Time Complexity Analysis. Include documentation for all implemented code using Doxygen. This should indicate the time complexity for each of the most relevant implemented algorithms.

4.2. Register Allocation using Graph Coloring

The project is structure as a series of progressively more complex algorithms, having at its core the graph-coloring algorithm described above. **The objective is to always use the minimum number of registers**. However, if you cannot color the interference graph with K colors (*i.e.*, perform the allocation with K registers) you either change (*i.e.*, simplify) the graph, the allowing it to be colored with that same number of register or increase the number of registers or perform a combination of both these approaches.

[T2.1: 4.0 points] Basic Register Allocation
In this basic approach, and given the input file specification, you are asked to implement and test the graph coloring and hence register allocation algorithm described above. Should the allocation be infeasible you should report it through a generic error message to the console in addition to the output file.

The algorithm indication in the input file in this case is **`basic`**, without any additional numeric parameter.

[T2.2: 3.0 points] Register Allocation with Web Spilling
Whenever the provided algorithm is **spilling** and should you not be able to color the graph with the basic algorithm, you may select up to **`K`** webs to spill where **`K`** is the numeric value provided with the algorithm

spilling indication in the input file.

The algorithm indication in the input file in this case is **`spilling`**, with a single additional numeric integer parameter separated by a comma as in "**`algorithm: spilling, 2`**".

Clearly, you should try to do graph coloring with as few as possible spilled webs, so the suggestion is that you start with one, only increasing the number of webs spilled if needed. Also, note that the choice of which web or webs you select for spilling greatly influences your ability to color the resulting graph. You should be clever about this selection. Describe the rationale you used in your algorithm implementation for the selection of which web(s) to spill.

[T2.3: 3.0 points] Register Allocation with Web Splitting
Whenever the provided algorithm is **splitting** and should you not be able to color the graph with the basic algorithm, you may select up to **`K`** webs to split where **`K`** is the numeric value provided with the algorithm splitting indication in the input file.

The algorithm indication in the input file in this case is **`splitting`**, with a single additional numeric integer parameter separated by a comma as in "**`algorithm: splitting, 2`**".

Clearly, you should try to do graph coloring with as few as possible splitted webs, so the suggestion is that you start with one, only increasing the number of webs splitted if needed. Also, note that the choice of which web or webs you select for splitting greatly influences your ability to color the resulting graph. You should be clever about this selection. Describe the rationale you used in your algorithm implementation for the selection of which web(s) to split.

[T2.4: 4.0 points] Register Allocation of Your Own
Whenever the provided algorithm is **free** you are free to pursue your own idea for register allocation, be it based on graph coloring or using any other approach. The only restriction is that web that interfere cannot be assign the same register.

The algorithm indication in the input file in this case is **`free`**, without any additional numeric integer parameter.

5. Demo & Presentation

[T3.1: 2.0 points] Giving a presentation that is "short-and-to-the point" is increasingly important. As such, you are required to structure a short 10-minute demo of your work, highlighting key aspects of your implementation, specifically:

- Present your solution for the provided input datasets demonstrating the results requested in this project;
- Highlight your graph class, explaining the conceptual decisions that were made, including alterations to the original structure provided;
- Highlight the most important and challenging aspects of your implementations.

You should also elaborate a PowerPoint presentation to support your presentation.

6. Turn-In Instructions & Deadline

Submit a zip file named **DA2026_PRJ2_T<TN>_G<GN>.zip** on Moodle, where TN stand for your TP number or "Turma" (as in T01, T02, etc.) and **GN** stands for your group number, with the following content:

- **Code** folder (contains program source code)
- Documentation folder (contains html documentation, generated using **Doxygen**)
- Presentation file (**PDF format**) that will serve as a basis for the demonstration.

Late submissions, up to 24 hours and 48 hours, will incur a penalty of 10% and 25% of the grade, respectively. No submissions will be accepted 48 hours after the deadline. Exceptions apply for justified and documented technical submissions issues.

Recall, the deadline is May 24, 2026, at 23:59 (PT time).

Appendix A. The 3-address Instructions

For the purposes of this assignment, you are to consider the following 3-address instructions all of which can be the target of a `jump` or `goto` instruction and therefore implicitly all have an associated label.

A.1. Generic Format and Symbols

The generic format of a basic 3-address instruction is simply: "`L: instr`" where the `instr` is an instruction that contains reserved keywords or operators alongside user-defined variable symbols or program label. It is assumed that labels have the generic form of "`L<n>`" where `<n>` is a positive integer. Clearly, no other class of symbols such as variables can have the same symbol as that of a label.

For simplicity in this assignment, **only integer values, integer constants and boolean values are used**. Whenever explicitly represented, the integer 0 (zero) value is equivalent to the Boolean false value whereas the non-zero value is equivalent to the true value. Real values (through the various floating-point representations) are not handled, neither are characters nor strings.

A.2. Arithmetic/Logic Instructions

These are generic two operand instructions such as "`a = b op c`" where `op` can be an arithmetic or logic operation defined over integers, Boolean or even real numbers. Obviously, this extends to unary operations such as logical or arithmetic complement as well as the simple assignment of variables such as "`a = b`" (see memory operations below). For complex expressions, compiler will insert temporary variables, with the appropriate names, for example, t0, or t1 that can hold intermediate expression results.

In every assignment the left-hand-side of the assignment must be a symbol that denotes a variable but one of the operands on the right-hand-side can include a constant value. Figure A2.1. below illustrates the "deconstruction" of a "complex" expression as a sequence of instruction to evaluate its value (under the assumed type requirements).

```
delta = (b*b) - (4*a*c);
```

(a)

```
t0 = 4 * a
t1 = t0 * c
t2 = b * b
t3 = t2 - t1
delta = t3
```

(b)

Figure A2.1. Example of a translation of a "complex" expression as a sequence of 3-address instructions.

A.3. Control-Flow

These are generic control-transfer instructions such as (un)conditional jumps. The conditional jump have the simple form of "`if pred goto L`" where `pred` is a simple predicate of the form "`a op b`" where types are compatible with the relational operator. Here `L` is a label of an instruction. More complex predicate can be developed by a sequence of instructions using logic operator such as `and`, `or` and `not`. Figure A3.1. below illustrates the "deconstruction" of a "complex" boolean predicate as a sequence of instruction to evaluate its value (under the assumed Boolean type requirements). As it can be seen, the 3-address instruction sequence follows an evaluate that is analogous to the one used for arithmetic expressions. This is not always the case as some language support and enforce "lazy evaluation".

```
if (a > b) and (x != 0) {
  <code F>
} else {
  <code T>
}
```

(a)

```
t0 = x != 0
t1 = a > b
t2 = t0 and t1
if t2 goto L1
  <instrs for code F>
  goto L2
L1:
  <instrs for code T>
goto L2
```

(b)

Figure A3.1. Example of a translation of a "complex" Boolean predicate expression as a sequence of 3-address instructions for a high-level conditional construct.

A.4. Procedure/Function Invocation and Return

A particular case of the `call` instructions, its arguments are set up on a stack (deliberately ill-defined in this representation) where values are stored (using the `putparam` instructions) and retrieved (using the `getparam` instruction). The `call` instruction itself has two parameters and can have a return value associated with it. In addition, it contains a numeric number indicating the number of arguments and correspondingly the number of `putparam` instructions that precede the `call` instruction itself.

The `ret` instruction may have a single operand for the case of procedure that returns a single value (in some languages there are called functions rather than procedures).

```
x = myAdd(x, y+1);

...

int myAdd(x,z){
  return x+z;
}
```

(a)

```
t0 = x
t1 = y + 1
putparam t1
putparam t0
t2 = call myAdd, 2
x = t2

...

myAdd:
  getparam x
  getparam z
  t3 = x + z
  ret t3
```

(b)

Figure A4.1. Example of a translation of a function call.

It is assumed that the actual arguments and formal parameters agree in number and type. Also, note that the order of the `putparam` and the corresponding `getparam` instructions is reversed as we are using a stack to hold the value in sequence.

For simplicity you may assume that all temporary variables and variable symbols are distinct irrespective of scope and that a procedure will terminate along its eventual control-flow path with a `ret` instruction without operand, whereas a function will terminate with a `ret` instruction with a single variable symbol, either one that corresponds to a variable or a temporary variable.

A.5. Array Manipulation and Bounds Checks

The array instructions have the simple form of "`A[<expr>]`" where "`<expr>`" is a single variable or a single constant expression (e.g., `A[i]` or `A[0]`) and `A` is a symbol referring to the array variable. Expressions such as `A[i+1]` are deconstructed by the compiler as a sequence of two instructions, namely, `t0 = i + 1` and `A[t0]`.

and `A` is a symbol associated with an array variable. Multi-dimensional arrays are not supported as they can be internally converted into linear structures with the appropriate indexing or memory address indirection. Figure A5.1 below illustrates the translation of a sequence of high-level instructions to 3-address intermediate representation involving array operations.

```
// --- swap values at
//     i and j+1 indices
tmp    = A[j+1];
A[j+1] = A[i];
A[i]   = tmp;
```

(a)

```
 t0 = j+1
tmp = A[t0]
t1  = i
A[t0] = A[t1]
A[t1] = tmp
```

(b)

Figure A5.1. Example of translation of high-level code to 3-address instructions.

One aspect of run-time safety is the possibility through expression manipulation to access an array beyond its boundaries. Depending on the language implementation this can be a source of malicious code injection and therefore a security hazard. For simplicity, you are to assume that the code is correct and no array is even accessed outside its declared range bounds.